Analysis and Design of Algorithms

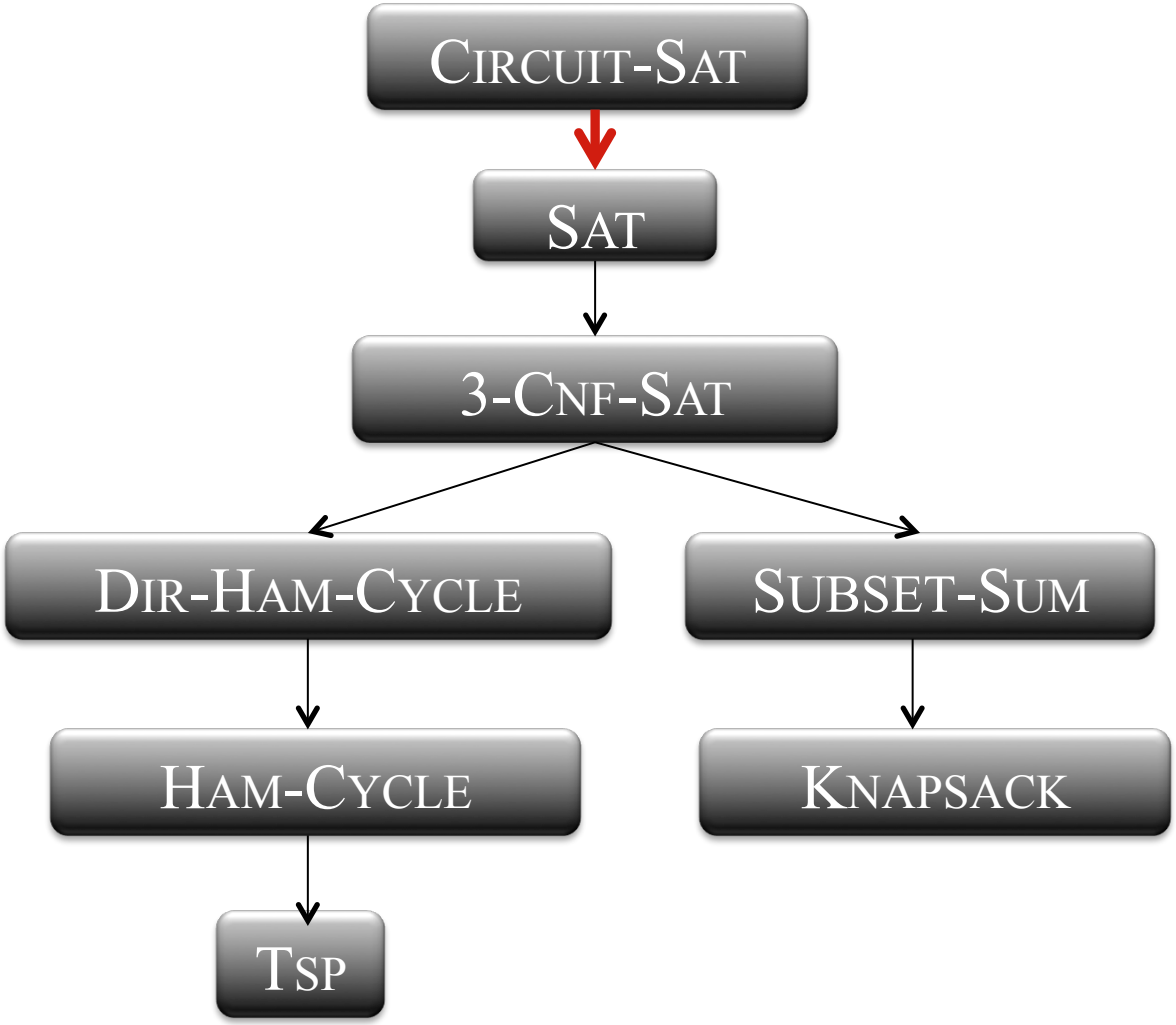
CS3230

NP-Completeness II

- Proving NP-completeness
- SAT
- 3-CNF-SAT
- Directed Hamiltonian cycle
- Hamiltonian cycle
- Travelling salesman
- Knapsack Problem

Prof Lee Wee Sun

NP-Complete Problems



Proving NP-Completeness

Lemma: Given a problem L, if $L' \leq_P L$ for some $L' \in NPC$, then L is ***NP-hard***. Moreover, if $L \in NP$, then $L \in NPC$

Proof:

Since $L' \in NPC$, we have $L'' \leq_P L'$ for all $L'' \in NP$. Hence, we have $L'' \leq_P L' \leq_P L$ for all $L'' \in NP$

Thus, L is ***NP-hard***. If, in addition, $L \in NP$, then $L \in NPC$

To prove that L is *NP-complete*

1. Find a suitable NP-complete problem L'
2. Show that $L' \leq_P L$. This shows that L is *NP-hard*
3. Show that L is in *NP*.

To show that $L' \leq_P L$

1. Provide a transformation that transform an instance $\alpha \in L'$ to an instance $\beta \in L$ in *polynomial time*.

2. Show that the solution to $\beta \in L$ also gives the solution to $\alpha \in L'$. Usually, we show that

- A *YES* solution to $\alpha \in L'$ implies a *YES* solution to $\beta \in L$, **and**
- A *YES* solution to $\beta \in L$ implies a *YES* solution to $\alpha \in L'$

Satisfiability (SAT)

Input:

- n boolean variables: $x_1, x_2, \ldots, x_n$
- A boolean formula φ formed by the Boolean variables, the operators, such as $\wedge$ (and), $\vee$ (or), $\neg$ (not), $\rightarrow$ (implication), $\leftrightarrow$ (equality), …, and parentheses

A *SAT* instance φ is satisfiable if there exists a truth assignment which causes φ to evaluate to 1

Output: *Yes* if φ is satisfiable; or *No* otherwise

Example 1: $\varphi = x_1 \wedge x_2 \wedge x_3 \wedge (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_3)$

φ is *satisfiable* by setting $x_1=1$, $x_2=1$, $x_3=1$

Example 2: $\varphi = (x_1 \leftrightarrow x_2) \wedge (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$

φ is *not satisfiable.*

$SAT \in NP$

Proof: Given a satisfying assignment, the verifying algorithm simply replaces each variable with the value as specified in the satisfying assignment, evaluates the formula in polynomial time, and check whether it outputs a 1.

Show that $\textsc{Circuit-Sat} \leq_P \textsc{Sat}$

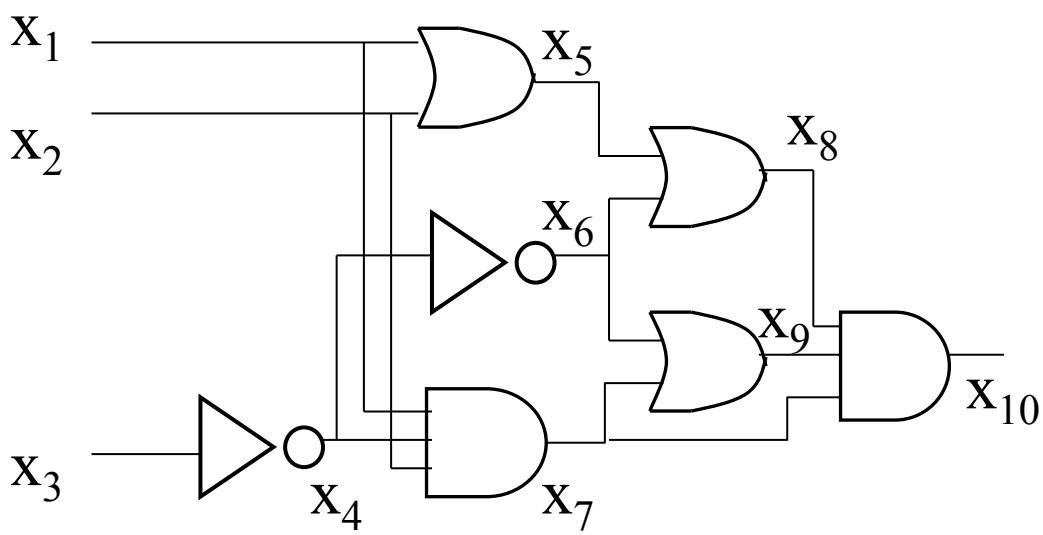
- Any instance of *circuit satisfiability* can be reduced in polynomial time to an instance of *formula satisfiability*
- Given a circuit N, we want to form a Boolean formula $f(N)$ such that N is satisfiable iff $f(N)$ is satisfiable
- We also need to ensure that $f(N)$ can be computed in polynomial time

Incorrect Reduction

Recursively expand the Boolean formula in a top-to-down fashion…

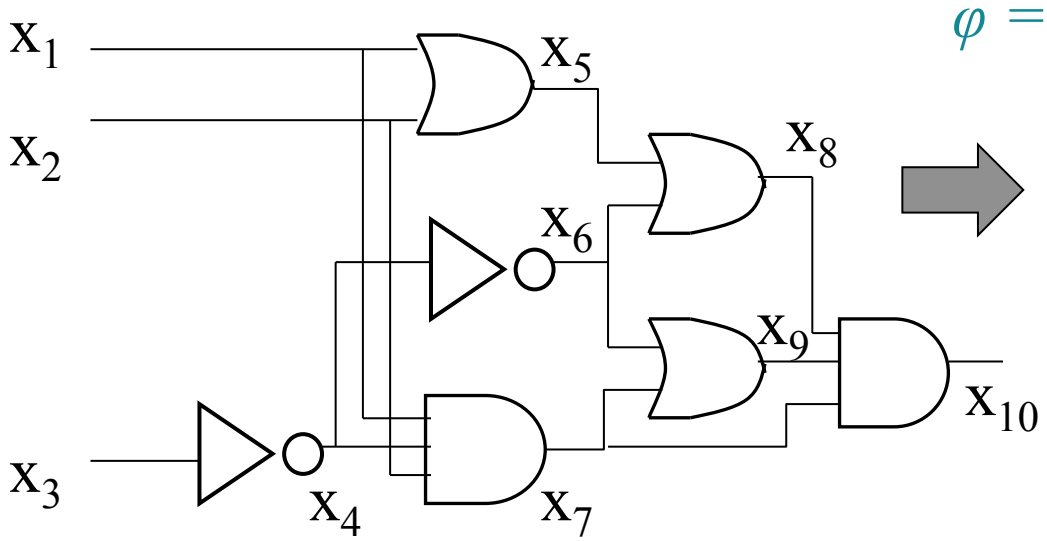
- $\varphi = x_{10} = x_7 \wedge x_8 \wedge x_9$
  $= (x_1 \wedge x_2 \wedge x_4) \wedge (x_5 \vee x_6) \wedge (x_6 \vee x_7)$
  $= \ldots.$

This transformation may generate exponential number of variables ☹



Correct Reduction

- Given a circuit N, for each gate G with input $y_1, y_2, \ldots, y_k$ and output z, we represent it as $z \leftrightarrow (y_1\ op\ y_2\ op\ \ldots\ op\ y_k)$ where op depends on the gate G ($\leftrightarrow$ is logical equality)
- We then *AND* the circuit output variable with the conjunction of clauses describing the operation of each gate



$$\varphi = x_{10} \land (x_{10} \leftrightarrow (x_7 \land x_8 \land x_9))$$
$$\land (x_9 \leftrightarrow (x_6 \lor x_7))$$
$$\land (x_8 \leftrightarrow (x_5 \lor x_6))$$
$$\land (x_7 \leftrightarrow (x_1 \land x_2 \land x_4))$$
$$\land (x_6 \leftrightarrow \neg x_4))$$
$$\land (x_5 \leftrightarrow (x_1 \lor x_2))$$
$$\land (x_4 \leftrightarrow \neg x_3)$$

- $\varphi = f(N)$ can be computed in polynomial time
- N is satisfiable iff $\varphi = f(N)$ is satisfiable
- Hence, $\textsc{Circuit-Sat} \leq_P \textsc{Sat}$

Recap:

To prove that a problem L is NP-complete, we

- ❑ Show that L belongs to NP
- ❑ Find a polynomial-time reduction to reduce a NP-complete problem to L

We showed that SAT is NP-complete by reduction from CIRCUIT-SAT

Analysis and Design of Algorithms

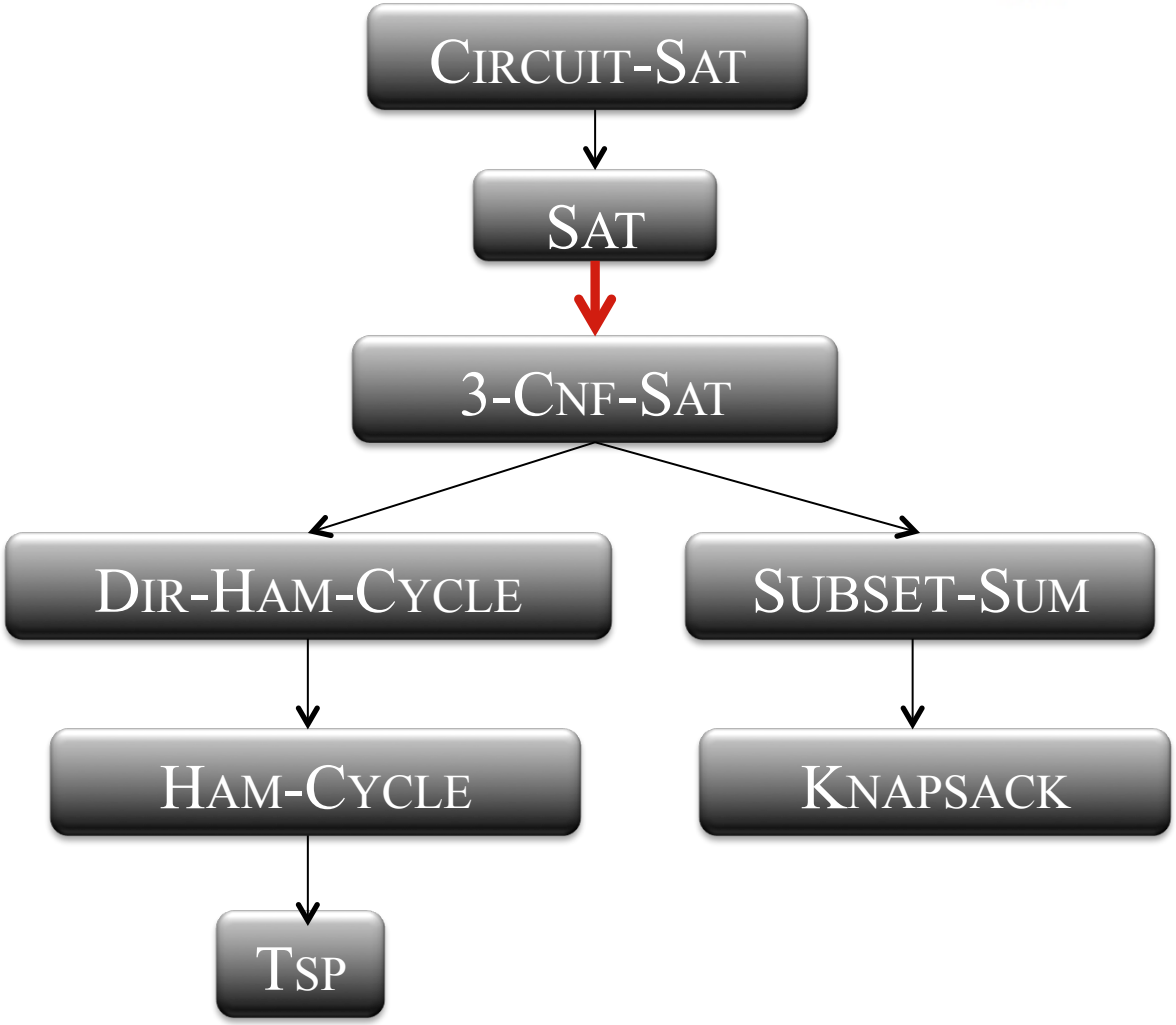
NP-Completeness II

- Proving NP-completeness
- SAT
- **3-CNF-SAT**
- Directed Hamiltonian cycle
- Hamiltonian cycle
- Travelling salesman
- Knapsack Problem

CS3230

Prof Lee Wee Sun

NP-Complete Problems



3-CNF-SAT

A ***literal*** in a Boolean formula is an occurrence of a ***variable*** or *its negation*.

CNF (Conjunctive Normal Form) is a Boolean formula expressed as

- *AND* of clauses, where
- Each clause is the *OR* of one or more *literals*

3-CNF is a CNF in which

- Each clause has *exactly 3 distinct literals*

3-CNF-SAT problem

Input: A 3-CNF formula φ

Output: Whether the 3-CNF formula φ is satisfiable?

Example 1: $\varphi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z)$

- Answer: Yes (certificate: $x=1$, $y=0$, $z=0$)

Example 2:

$$\varphi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee \neg y \vee \neg z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee \neg z)$$

- Answer: No

3-CNF-SAT $\in$ NP

The **certificate** is the assignment of 0 and 1 for the variables. Given the assignment of variables we can evaluate in polynomial time if the 3-CNF boolean formula is satisfiable

Example: Given the certificate $x=1$, $y=0$, $z=0$, we can show that

$$\varphi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z) = 1$$

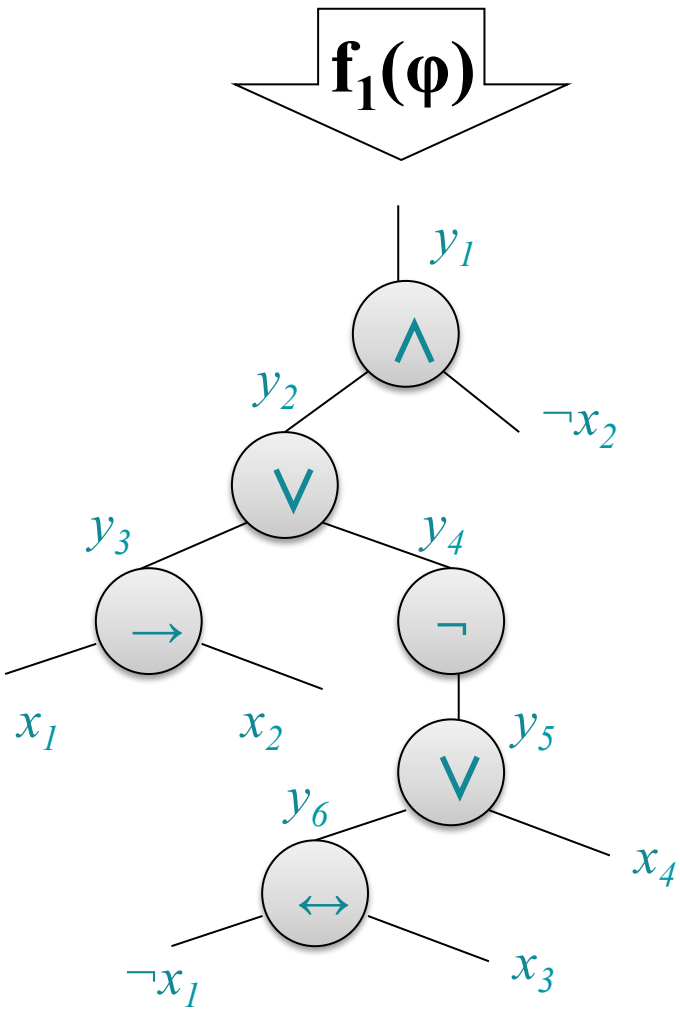
Hence, *3-CNF-SAT* $\in$ *NP*

SAT $\leq_P$ 3-CNF-SAT

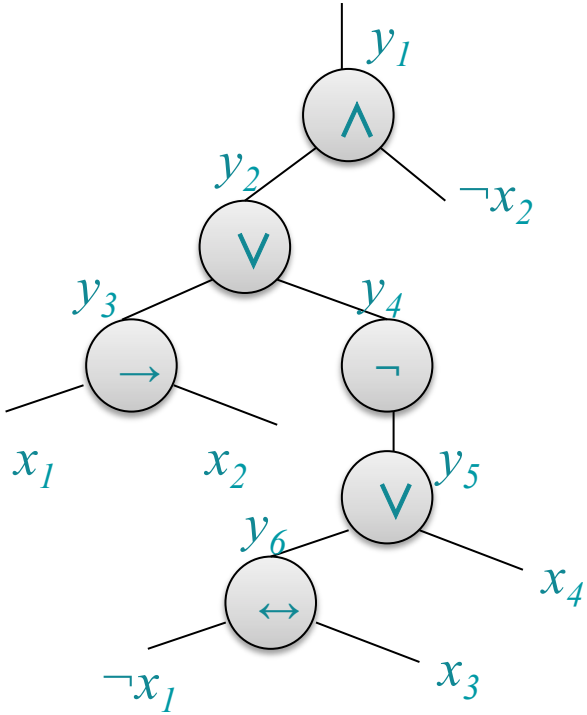
Given a Boolean formula φ, we will give a two-step algorithm to obtain a *3-CNF* $\varphi'' = f_2(f_1(\varphi))$

- **Step 1:** $\varphi' = f_1(\varphi)$ generates a Boolean formula expressed as *AND* of clauses where each clause has 3 literals
  φ' is satisfiable iff φ is satisfiable
- **Step 2:** $\varphi'' = f_2(\varphi')$ converts every clause in φ' into *3-CNF*
  φ'' is satisfiable iff φ' is satisfiable

$$\varphi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$



1. We construct a binary ***parse tree*** of φ, ***literals*** **as** ***leaves*** and ***connectives*** **as** ***internal vertices***
2. We introduce a variable y_i for the output of each internal vertices



$$\varphi' = f_1(\varphi) = y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$$
$$\wedge (y_2 \leftrightarrow (y_3 \vee y_4))$$
$$\wedge (y_4 \leftrightarrow \neg y_5)$$
$$\wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2))$$
$$\wedge (y_5 \leftrightarrow (y_6 \vee x_4))$$
$$\wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))$$

3. We rewrite the formula to φ' as the *AND* of the root variable and a conjunction of clauses describing the operation of each vertex
   - Similar to CIRCUIT-SAT $\leq_P$ SAT reduction
   - In φ', each clause has at most three literals

$f_2(\varphi')$

We change each clause φ_i' into CNF as follows:

1. Construct a truth table for φ_i'
   - This table will be small, at most $2^3=8$ rows by 4 columns
2. Write the ***Disjunctive Normal Form (DNF)*** for all truth table items that evaluate to 0
   - We will later negate this back using ***DeMorgan's law*** (step 3), that's why we need items that evaluate to 0 for now

3. By using DeMorgan's law, we obtain the CNF

- $\neg(a \wedge b) = \neg a \vee \neg b$
- $\neg(a \vee b) = \neg a \wedge \neg b$

Converting $\varphi_2' = y_1 \leftrightarrow (y_2 \wedge \neg x_2)$ into 3-CNF

y1	y2	x2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Disjunctive Normal Form:

$$\varphi_2' = (y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2)$$

Conjunctive Normal Form:

$$\varphi_2'' = (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)$$

The resulting clauses are in *CNF* and each clause has 3 (or fewer) literals

For the final step, we change 1 or 2-literal clause into 3-literal clause as follows, introducing new variables p and q as required:

- If a clause has one literal l, change it to $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$
- If a clause has two literals $(l_1 \vee l_2)$, change it to $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$

3-CNF-SAT is NP-complete

Given a Boolean formula φ, we have shown that we can reduce it to a 3-CNF formula φ'' by (1) $\varphi' = f_1(\varphi)$ and then (2) $\varphi'' = f_2(\varphi')$

Note that:

- φ is satisfible iff φ' is satisfible iff φ'' is satisfible
- Also, the reduction takes polynomial time

Hence, 3-CNF-SAT is NP-hard. As 3-CNF-SAT is in NP, 3-CNF-SAT is **NP-complete**

Analysis and Design of Algorithms

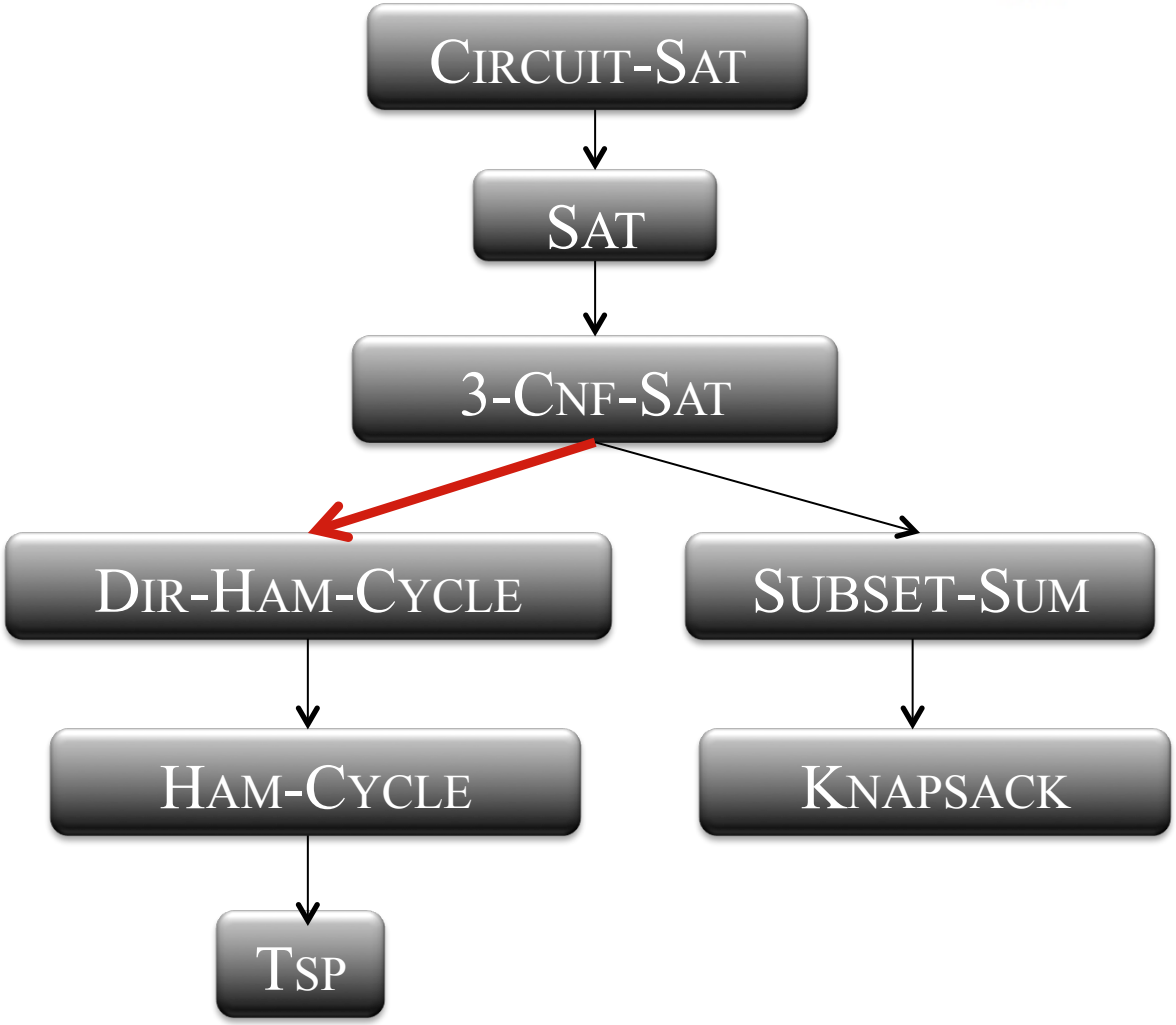
CS3230

Prof Lee Wee Sun

NP-Completeness II

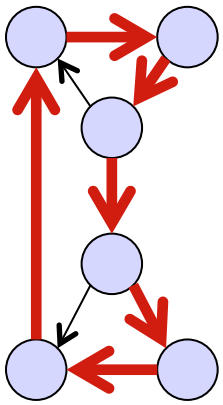
- Proving NP-completeness
- SAT
- 3-CNF-SAT
- **Directed Ham cycle**
- Hamiltonian cycle
- Travelling salesman
- Knapsack Problem

NP-Complete Problems

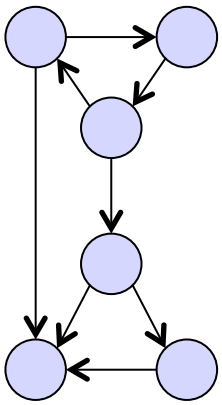


Directed Hamiltonian Cycle

DIR-HAM-CYCLE: Given a digraph $G = (V, E)$, does there exists a simple directed cycle Γ that contains every node in V exactly once?



Has Hamiltonian cycle



No Hamiltonian cycle

DIR-HAM-CYCLE $\in$ NP

Given a directed graph $G=(V,E)$ and a certificate, a cycle $C=(v_1, v_2, \ldots, v_n, v_1)$, the verifying algorithm checks if:

1. C has each vertex in V exactly once
2. The first and the last vertex in C is the same (a cycle)
3. All consecutive vertices $(v_1, v_2), (v_2, v_3), \ldots, (v_{n-1}, v_n), (v_n, v_1) \in E$

This check can be done in $O(V+E)$, i.e. polynomial time.

3-CNF-SAT $\leq_p$ DIR-HAM-CYCLE

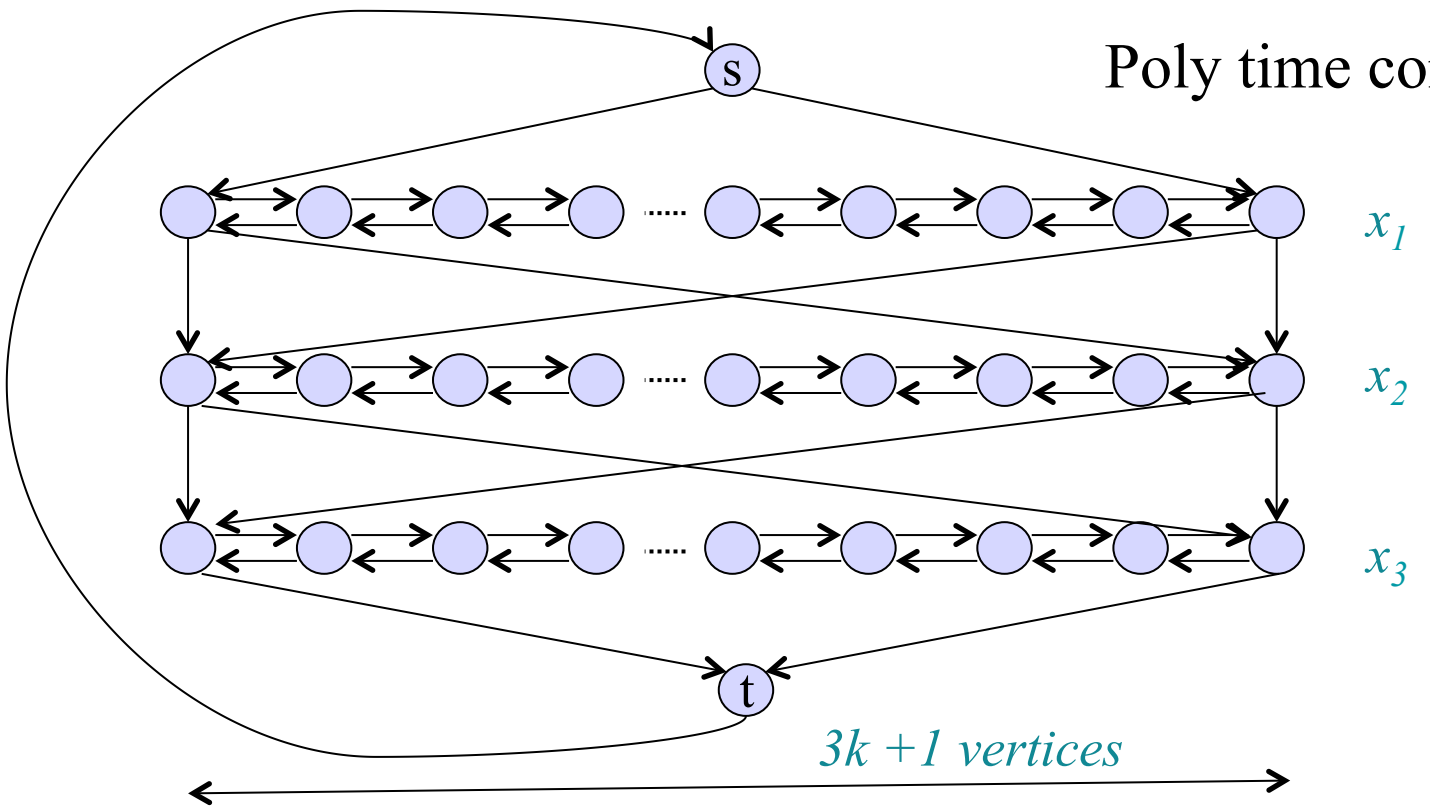
Polynomial time transformation:

Given an instance Φ of *3-CNF-SAT*, we construct an instance of *DIR-HAM-CYCLE* that has a Hamiltonian cycle iff Φ is satisfiable.

Given *3-SAT* instance Φ with n variables x_i and k clauses, *first* construct a graph with

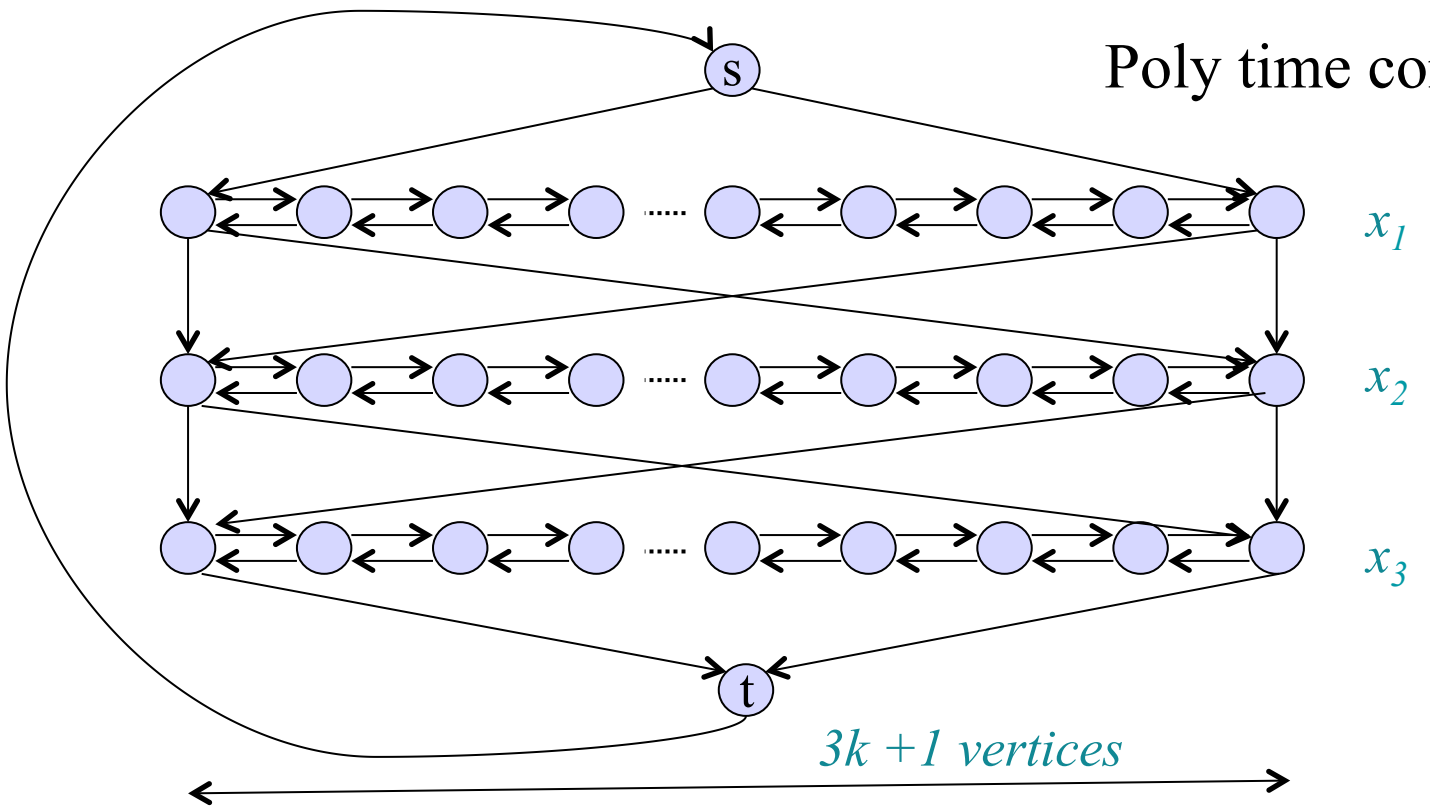
- a start vertex s, a terminal vertex t

Poly time construction



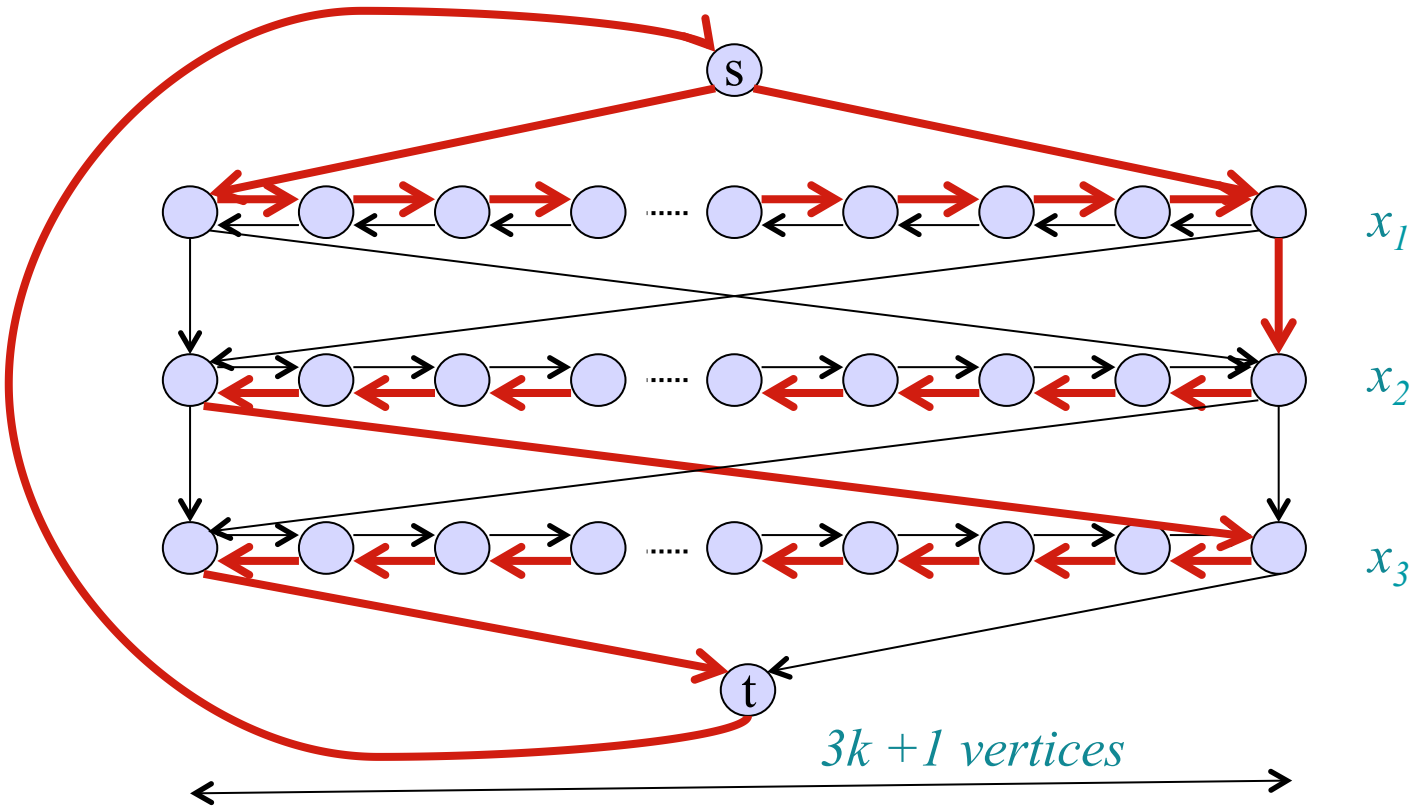
- n rows of vertices corresponding to variables, and
- $3k$ sets of bidirectional edges between $3k+1$ columns of vertices corresponding to k clauses.

Poly time construction

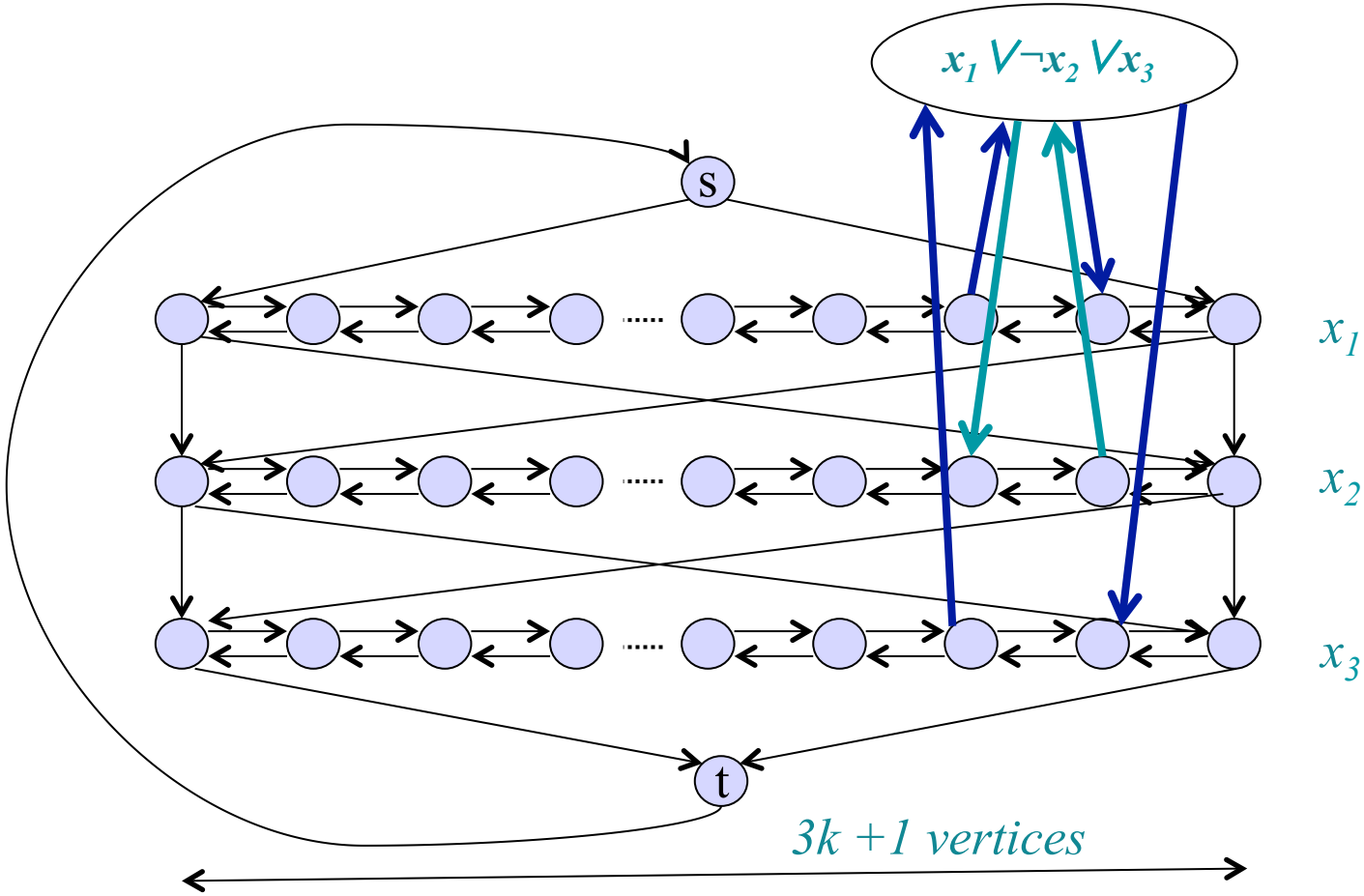


There are 2^n Hamiltonian cycles corresponding to moving right on row i when $x_i = 1$ and moving left otherwise. Example below:

$x_1=1, x_2=0, x_3=0$

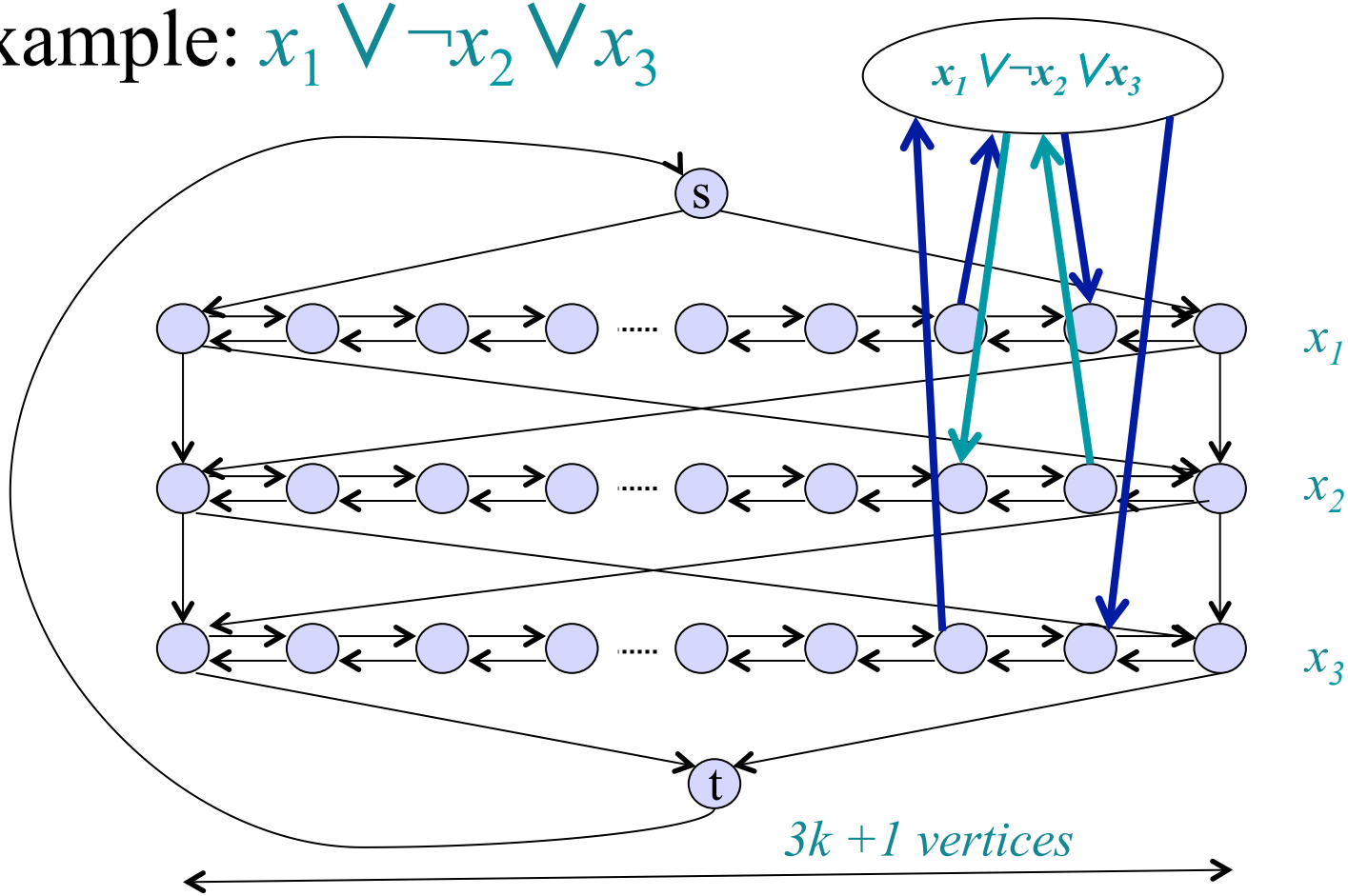


Then add a vertex for each clause (corresponding to the middle of its 3 columns of edges) to enforce the clause constraints



- If clause contains x_i edge set directions towards right
- If clause contains $\neg x_i$ edge set directions towards left

Example: $x_1 \vee \neg x_2 \vee x_3$



If Φ is satisfiable then G has a Ham cycle.

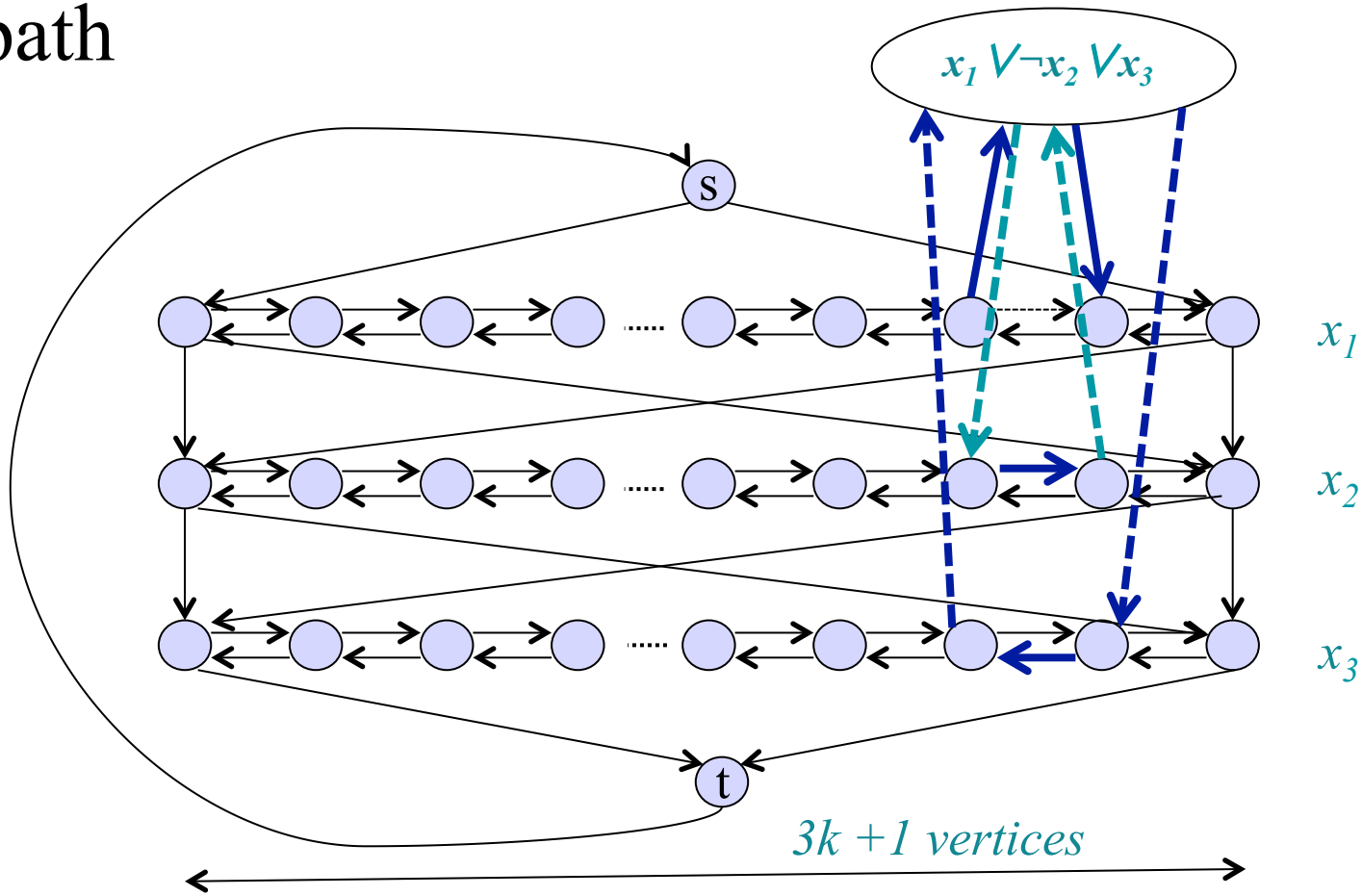
Proof:

Suppose the *3-CNF-SAT* instance has satisfying assignment x^*. Then, define Hamiltonian cycle in G as follows:

- if $x^*_i = 1$, traverse row i from *left to right*
- if $x^*_i = 0$, traverse row i from *right to left*

As the formula is satisfied by the assignment, for each clause C_j, there will be at least one row i in which we are going in correct direction allowing vertex C_j to be spliced in.

Example: $x_1=1$, $x_2=1$, $x_3=0$ splices in vertex $x_1 \vee \neg x_2 \vee x_3$ by taking the solid, rather than dotted path

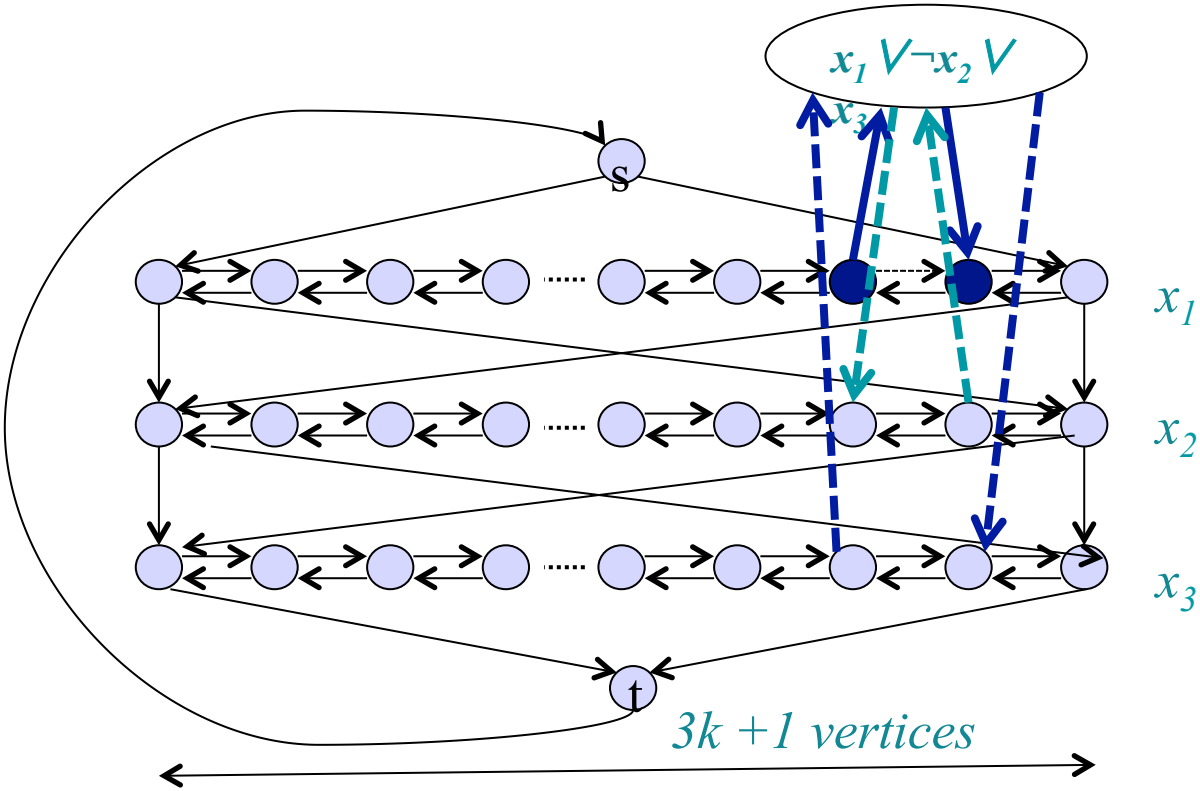


If G has a Ham cycle, then Φ is satisfiable.

Proof:

Suppose G has a Hamiltonian cycle Γ.

First we argue that if an edge enters a clause vertex through a particular row, it will exit the clause vertex through the same row.

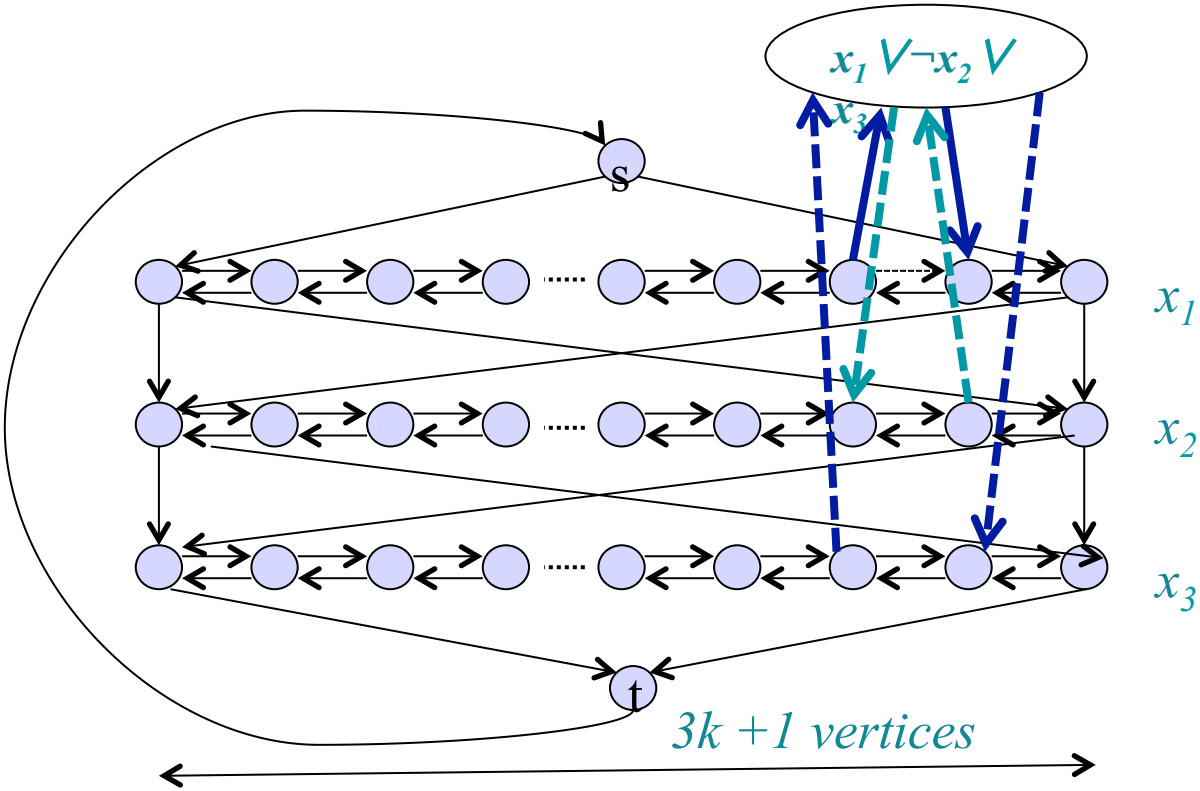


Assume that the cycle enters the clause at the highlighted (left) vertex. If it does not exit the clause through the highlighted vertex next to it, when it visits that vertex from the right it would be unable to proceed further.

We construct an assignment A as follows:

- If row i is traversed from left to right, set x_i to 1
- Otherwise set x_i to 0

We now argue that every clause is satisfied by the assignment A.



Consider a clause vertex C_j where the incoming edge in the Hamiltonian cycle comes from the left (respectively right) on row k.

- By construction, $x_k = 1$ (resp. $x_k = 0$), as assigned by A satisfies the clause.

Recap:

We prove that DIR-HAM-CYC is NP-complete by reduction from 3-CNF-SAT

- ❑ Prove that DIR-HAM-CYC $\in$ NP
- ❑ Prove 3-CNF-SAT $\leq_p$ DIR-HAM-CYCLE
  - ❑ Construct a graph from the formula
  - ❑ Prove that if the formula is satisfiable then there is a directed Hamiltonian cycle
  - ❑ Prove that if there is a directed Hamiltonian cycle then we can get a satisfying assignment.

Analysis and Design of Algorithms

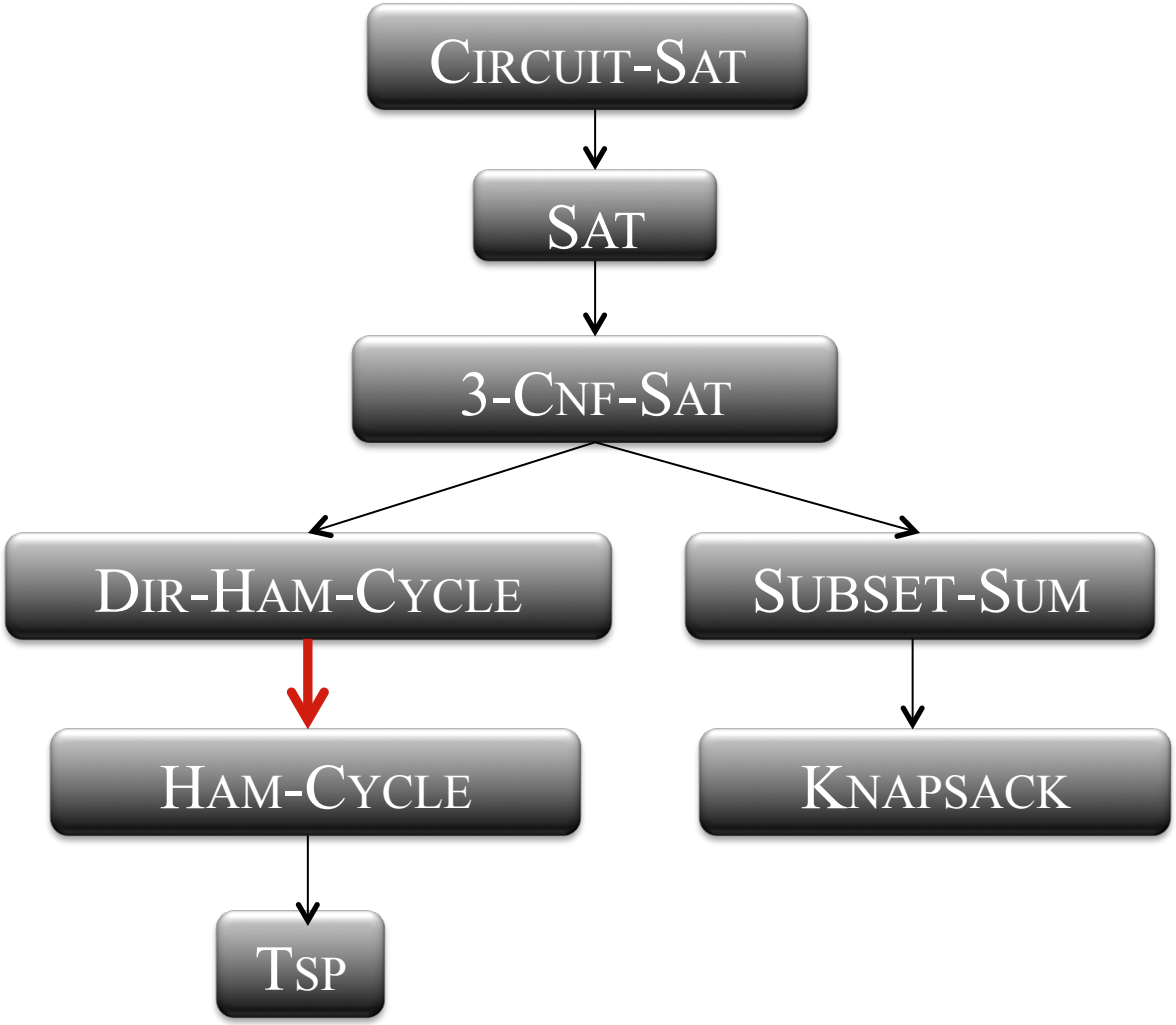
CS3230

Prof Lee Wee Sun

NP-Completeness II

- Proving NP-completeness
- SAT
- 3-CNF-SAT
- Directed Hamiltonian cycle
- **Hamiltonian cycle**
- Travelling salesman
- Knapsack Problem

NP-Complete Problems



Hamiltonian Cycle

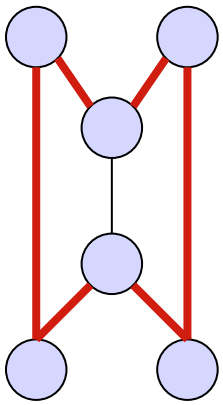
HAM-CYCLE

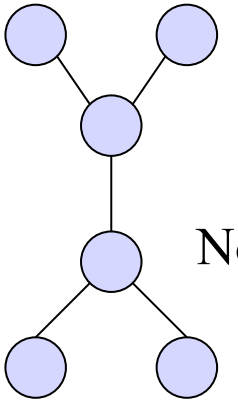
Input: An undirected graph $G=(V,E)$

- A Hamiltonian cycle is a simple cycle that contains all vertices in V

Output: Decide if G contains a Hamiltonian cycle

Hamiltonion cycle (shown in **red**)





No Hamiltonion cycle

HAM-CYCLE $\in$ NP

Given an undirected graph $G=(V,E)$ and a certificate, a cycle $C=(v_1, v_2, \ldots, v_n, v_1)$, the verifying algorithm checks if:

- C has each vertex in V exactly once
- The first and the last vertex in C is the same (a cycle)
- All consecutive vertices $(v_1, v_2), (v_2, v_3), \ldots, (v_{n-1}, v_n), (v_n, v_1) \in E$

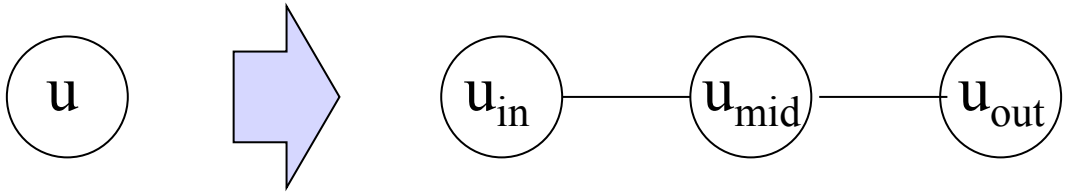
This check can be done in $O(V+E)$, i.e. polynomial time. Hence, *HAM-CYCLE* $\in$ *NP*

DIR-HAM-CYCLE $\leq_P$ HAM-CYCLE

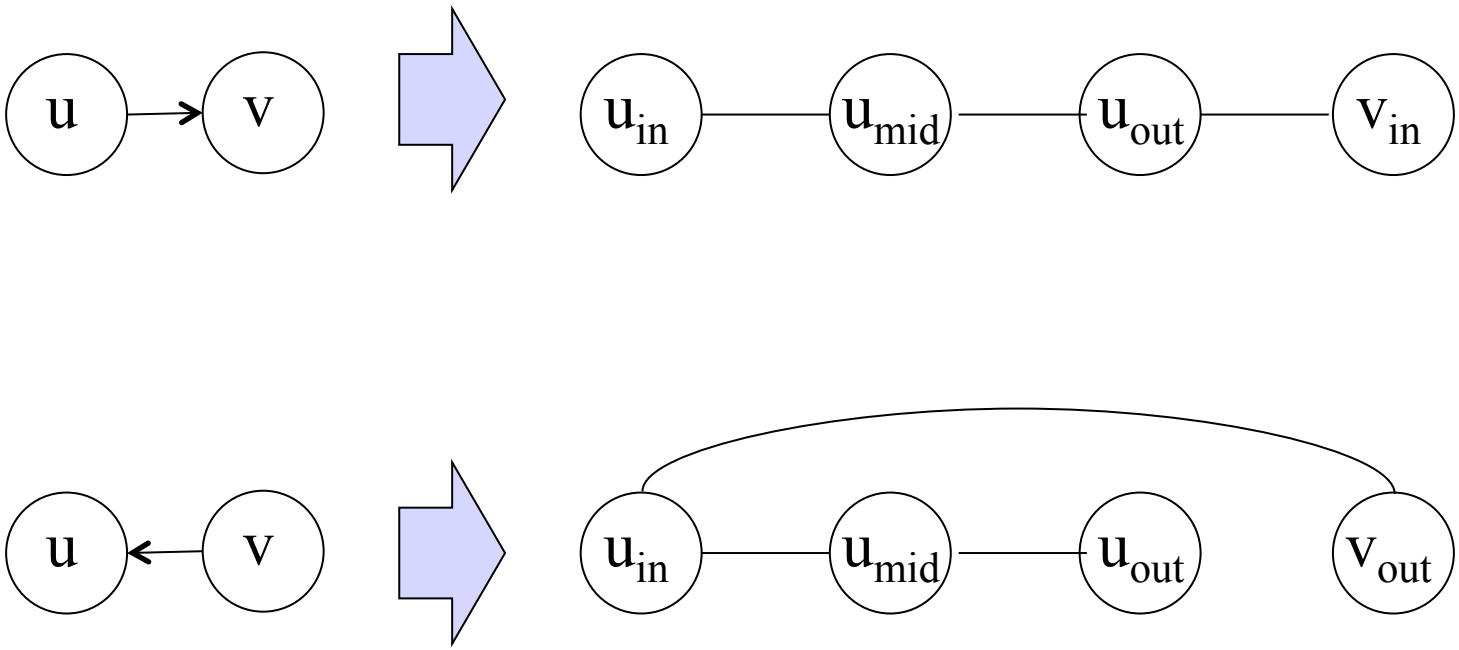
Polynomial time transformation:

Given a digraph $G = (V, E)$, construct G'

- Convert vertex u into three vertices u_{in}, u_{mid}, u_{out} with two undirected edges (u_{in}, u_{mid}), (u_{mid}, u_{out}), and



- Convert every directed edge (u,v) into (u_{out}, v_{in})
- Clearly polynomial time.


u
v
u_{in}
u_{mid}
u_{out}
v_{in}
u
v
u_{in}
u_{mid}
u_{out}
v_{out}

If G has a Hamiltonian cycle, then G' has a Hamiltonian cycle

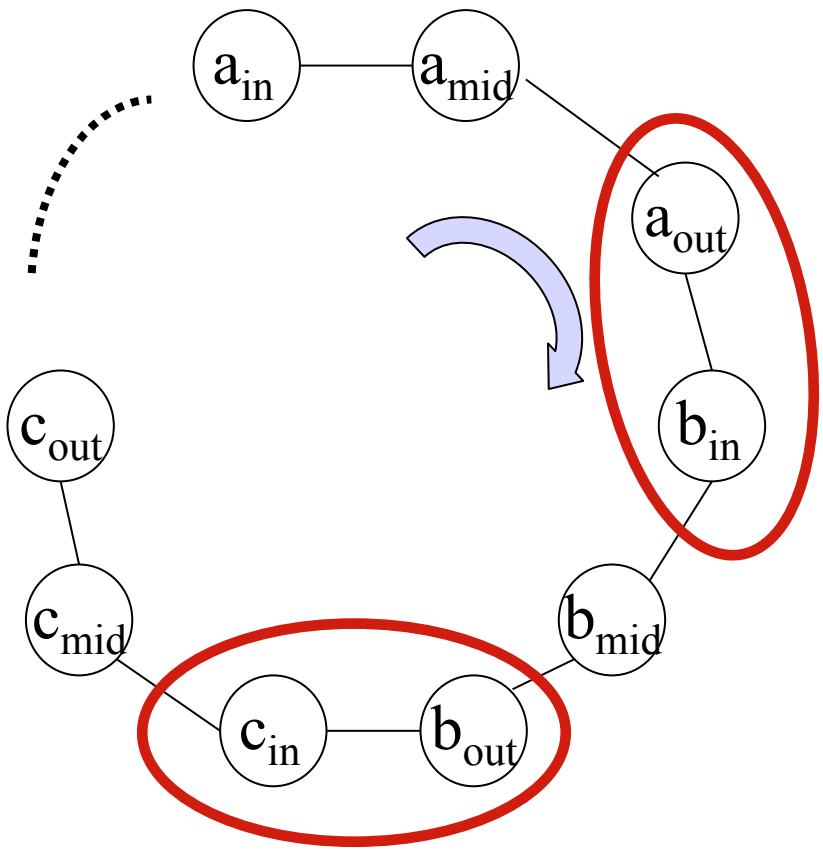
Proof:

- For each edge (u,v) in the cycle in G, extend the path in G' by (u_{out}, v_{in}), (v_{in}, v_{mid}), (v_{mid}, v_{out}).
- Every vertex u appears once in the cycle, hence every triple u_{in}, u_{mid}, u_{out} appears once in the construction.
- From construction, the path eventually forms a cycle.
- Hence G' has a Hamiltonian cycle.

If G' has a Hamiltonian cycle, then G has a Hamiltonian cycle

Proof:

- Select every edge (u_{out}, v_{in}) in sequence in the cycle and relabel it (u,v)
  - By construction this corresponds to an edge (u,v) in G.



- This gives a Hamiltonian cycle in G

Recap:

We prove that HAM-CYC is NP-complete

- ❑ Prove that HAM-CYC $\in$ NP
- ❑ Prove DIR-HAM-CYCLE $\leq p$ HAM-CYCLE
  - ❑ Construct an undirected graph G' from the directed graph G
  - ❑ Prove that a Ham-cycle G implies a Ham-cycle in G'
  - ❑ Prove that a Ham-cycle G' implies a Ham-cycle in G

Analysis and Design of Algorithms

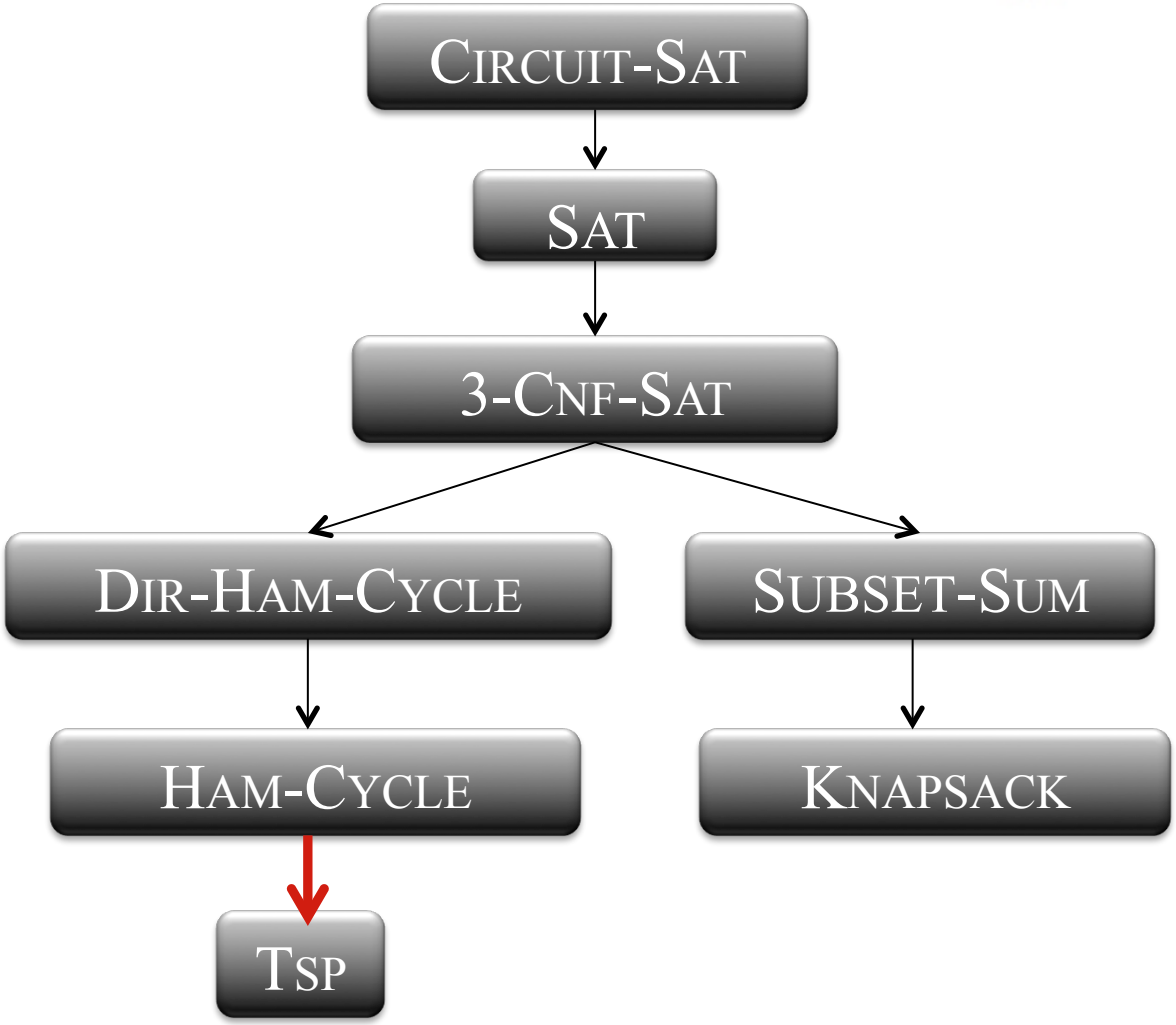
CS3230

NP-Completeness II

- Proving NP-completeness
- SAT
- 3-CNF-SAT
- Directed Hamiltonian cycle
- Hamiltonian cycle
- **Travelling salesman**
- Knapsack Problem

Prof Lee Wee Sun

NP-Complete Problems



Travelling Salesman Problem

TSP

Input:

- A complete directed graph $G=(V, V \times V)$
- A cost function $c: V \times V \rightarrow Z$
- A cost threshold k

Output: Decide if there exists a *tour* that visits all vertices in V with cost at most k

TSP $\in$ NP

Given a certificate, the tour T: $(v_1, v_2, \ldots, v_n, v_1)$, check if:

1. T has each vertex in V exactly once
2. The *first* and the *last* vertex in T is the same (a cycle).
3. $c(v_n, v_1) + \Sigma_{i=1..n-1}\, c(v_i, v_{i+1}) \leq k$

This is can be done in linear (polynomial) time. Hence, $TSP \in NP$

Hamiltonian Cycle

HAM-CYCLE

Input: An undirected graph $G=(V,E)$

- A Hamiltonian cycle is a simple cycle that contains all vertices in V

Output: Decide if G contains a Hamiltonian cycle

HAM-CYCLE $\leq_P$ TSP

Transformation:

- Input: $H=(V, E)$
- Set $G=(V, V \times V)$
  - $c(v,u) = c(u,v) = 0$ if $(u,v) \in E$, and 1 otherwise
- Set $k = 0$

Clearly polynomial time

If H has a Hamiltonian cycle then G has a tour with cost 0

Proof: Let $(v_1, v_2, \ldots, v_n, v_1)$ be the Hamiltonian cycle in H

Then, $(v_1, v_2, \ldots, v_n, v_1)$ is a cost-0 tour in G

- Reason: $(v_i, v_{i+1}) \in E \rightarrow c(v_i, v_{i+1})=0$ for $i=1, 2, \ldots, n-1$
- Also, the last edge $(v_n, v_1) \in E \rightarrow c(v_n, v_1)=0$

If G has a tour with cost 0 then H has a Hamiltonian cycle

Proof: Let $(v_1, v_2, \ldots, v_n, v_1)$ be a cost-0 tour in G. Then, $(v_1, v_2, \ldots, v_n, v_1)$ is a Hamiltonian cycle in H.

- Since the cost of the tour is 0, we have $c(v_n, v_1)=0$ and $c(v_i, v_{i+1})=0$ for $i=1, 2, \ldots, n-1$
- $c(v_i, v_{i+1})=0 \rightarrow (v_i, v_{i+1}) \in E$ for $i=1, 2, \ldots, n-1$
- Also, $c(v_n, v_1)=0 \rightarrow (v_n, v_1) \in E$
- Hence, $(v_1, v_2, \ldots, v_n, v_1)$ is a Hamiltonian cycle in H

Recap:

Prove TSP is NP-complete

- Prove that TSP $\in$ NP
- Prove that HAM-CYCLE $\leq_p$ TSP
  - Construct a complete directed graph G from the directed graph H with appropriate edge costs
  - Prove that a Ham-cycle in H implies a zero cost tour in G
  - Prove that a zero cost tour in G implies a Ham-cycle in H

Analysis and Design of Algorithms

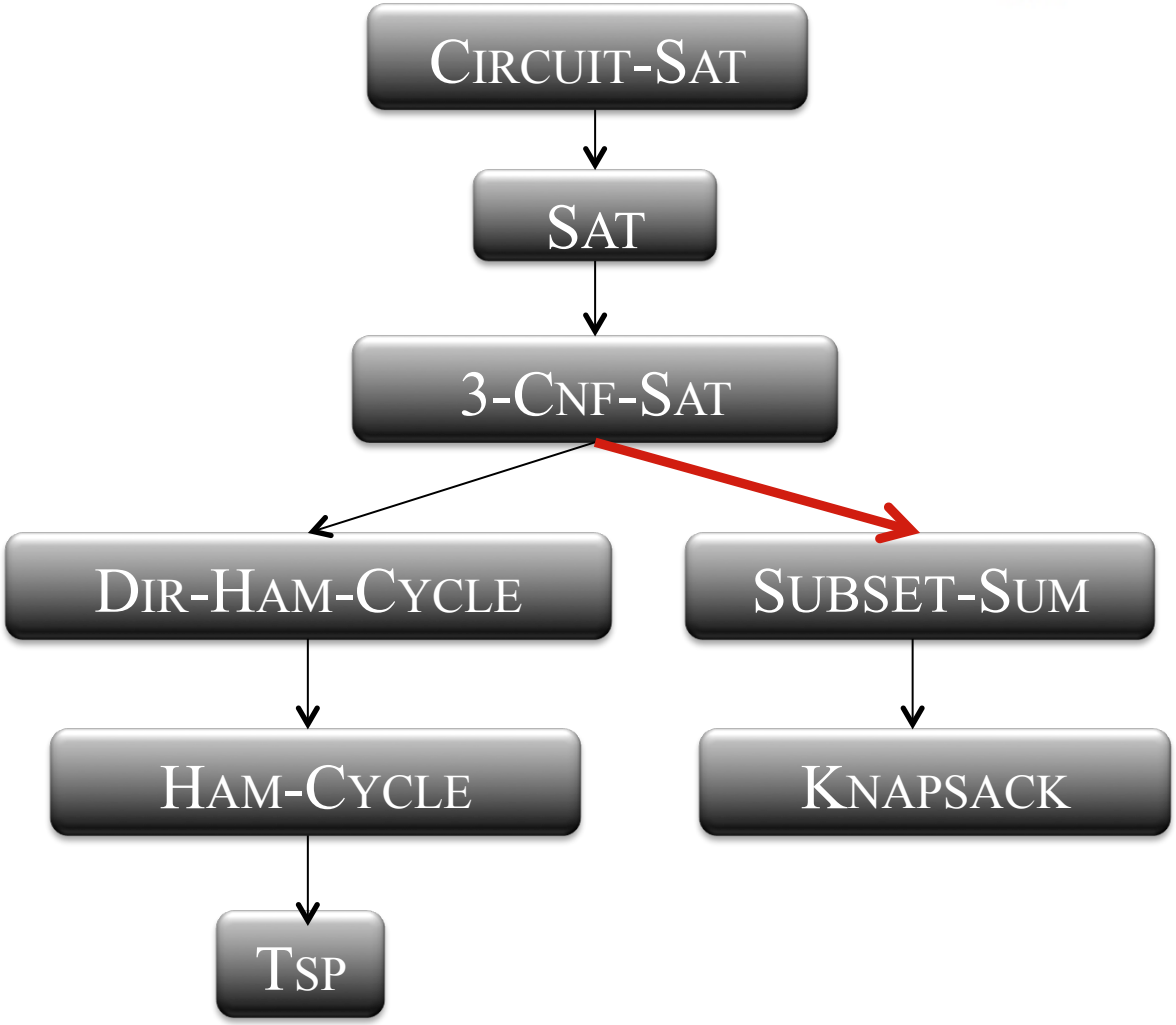
NP-Completeness II

- Proving NP-completeness
- SAT
- 3-CNF-SAT
- Directed Hamiltonian cycle
- Hamiltonian cycle
- Travelling salesman
- **Knapsack Problem**

CS3230

A/P Lee Wee Sun

NP-Complete Problems



SUBSET-SUM

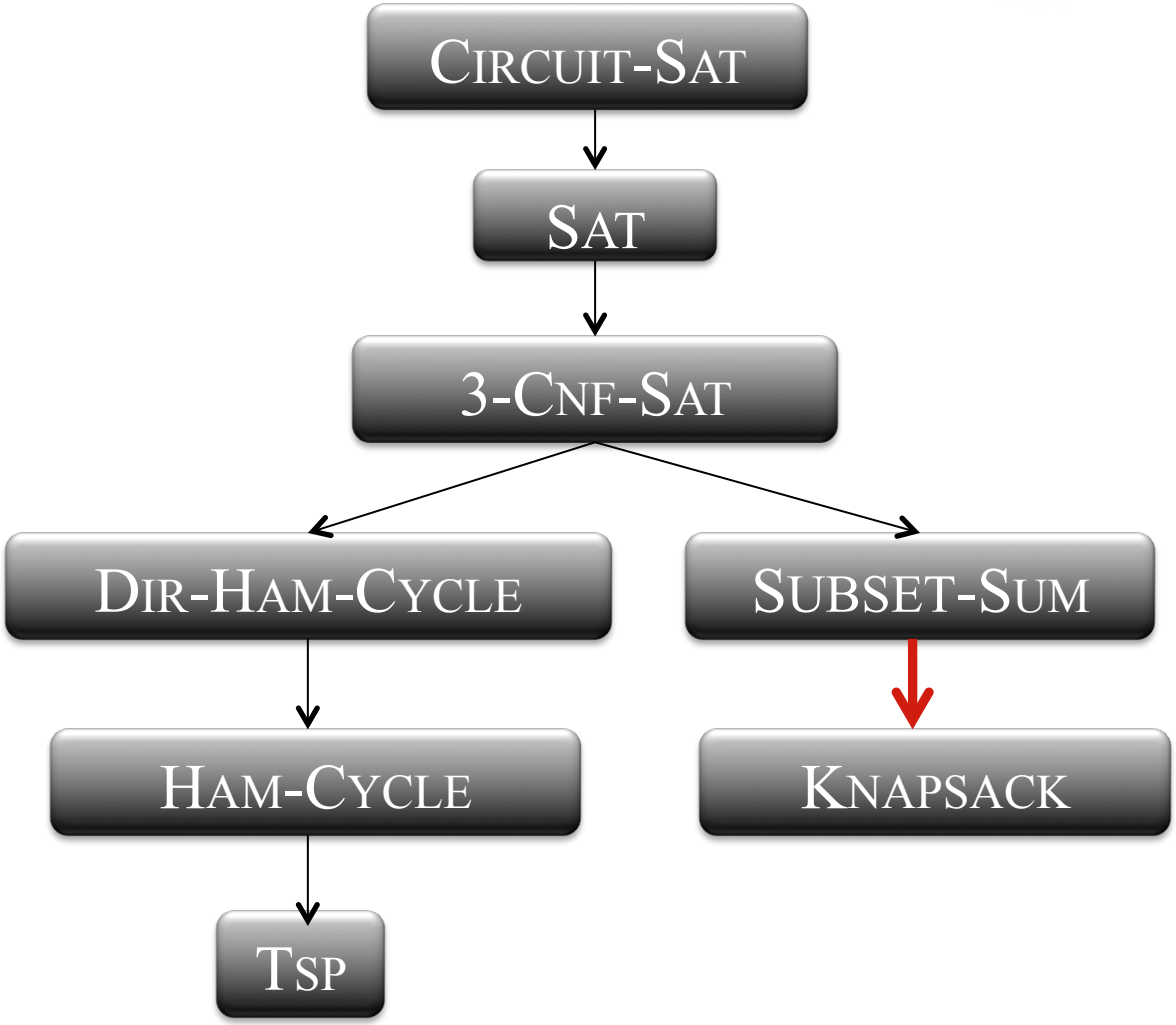
Input: A set of integers S and an integer t

Output: Decide if there is a non-empty subset $S' \subseteq S$ such that $\sum_{x \in S'} x = t$

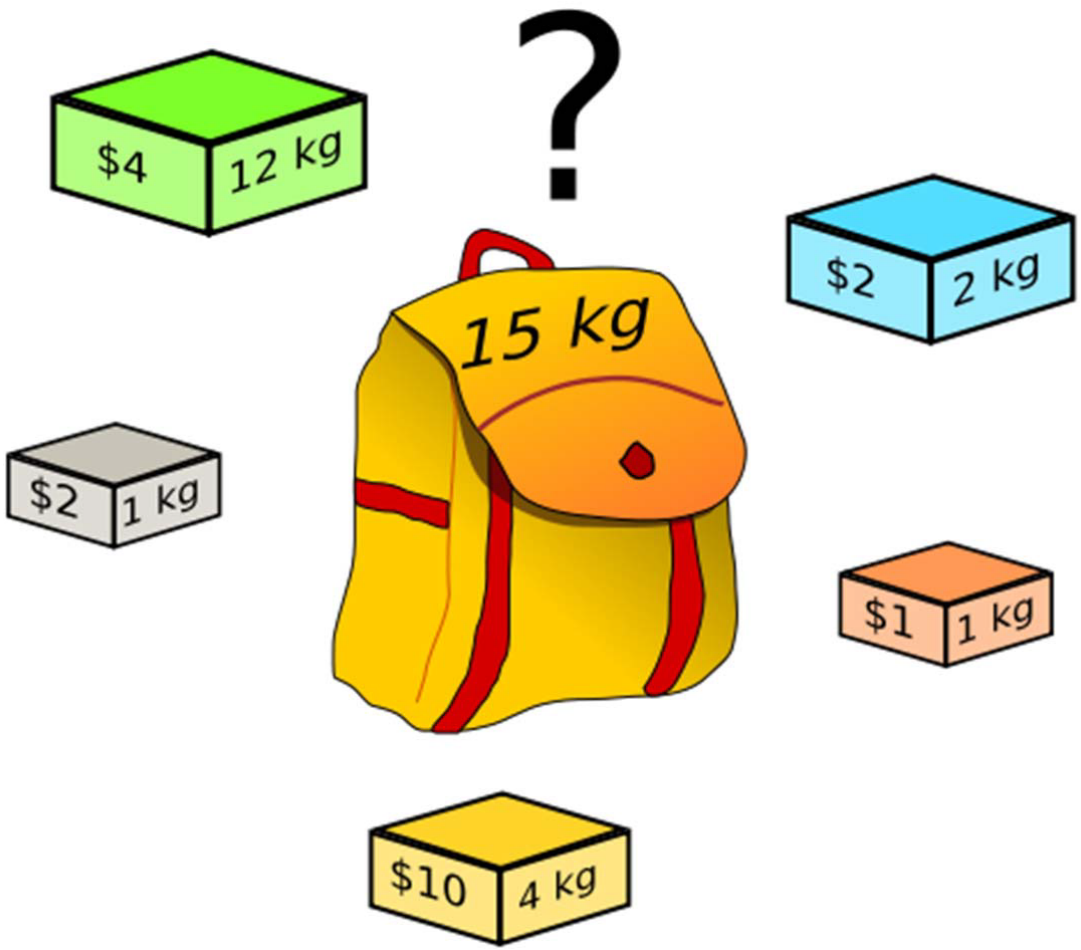
3-CNF-SAT $\leq_P$ SUBSET-SUM

The reduction can be found in CLRS Chapter 34.5.5.

NP-Complete Problems



Knapsack Problem



What is the maximum value you can get?

$15

Knapsack Problem

KNAPSACK

Input:

- $(w_1, v_1), (w_2, v_2), \ldots, (w_n, v_n)$ where w_i and v_i are non-negative integers
- Capacity W
- Value threshold V

Output: Is there a subset of items with total weight at most W and value at least V ?

KNAPSACK $\in$ NP

Given a certificate, a subset of items, the verifying algorithm checks if:

1. The total weight of the subset if no more than W
2. The total value of the subset is at least V

This is doable in linear (polynomial) time,
Hence, $KNAPSACK \in NP$

SUBSET-SUM $\leq_P$ KNAPSACK

Transformation:

- Given a SUBSET-SUM problem S with non-negative numbers $s_1, \ldots, s_n$ and target t, create a knapsack problem K with
  - $v_i = w_i = s_i$
  - $W = V = t$

This can clearly be done in polynomial time

If there is subset in *SUBSET-SUM* problem P that sums to t, then there is a solution to the knapsack problem K

- The solution to the subset sum problem P gives a subset S' such that $\Sigma_{i \in S'} s_i = t$
- For problem K, we have $v_i = w_i = s_i$ and $W = V = t$. This gives

  $\Sigma_{i \in S'} w_i = t = W$ and $\Sigma_{i \in S'} v_i = t = V$
- Hence S' is a solution to K

If there is a solution to the knapsack problem K then there is subset in *SUBSET-SUM* problem P that sums to t

- The solution to K gives

  $\Sigma_{i \in S'} w_i \leq W$ and $\Sigma_{i \in S'} v_i \geq V$

- From the transformation we have $v_i = w_i = s_i$ and $W = V = t$ giving

  $\Sigma_{i \in S'} s_i \leq t$ and $\Sigma_{i \in S'} s_i \geq t$ implying $\Sigma_{i \in S'} s_i = t$

- Hence S' is a solution to the subset sum problem P

Recap

- Proving NP-completeness
- SAT
- 3-CNF-SAT
- Directed Hamiltonian cycle
- Hamiltonian cycle
- Travelling salesman
- Knapsack Problem